Topological Score Grammar — Implementation Companion

DSO v4.2: review synthesis, SuperCollider/Python sound capabilities, the crossfade-emergence design change, and reference tables for every pattern, schema, state element, module, rule, weight and elected signal implemented in this release

1 Sources reviewed and what the grammar keeps

The shared Google Drive folder could not be read in this session (the connector exposes only share/rename/trash, no listing or fetch); the synthesis below draws on the established reading stack: Lerdahl & Jackendoff (GTTM), Lerdahl (*Tonal Pitch Space*), Temperley, Hasty, London, Huron, Meyer, Cohn, Lewin, Gjerdingen, Toussaint, McLean & Wiggins (Tidal), Di Scipio, Xenakis, Blackwell & Young. Drop the Drive texts into the project files and this section can be extended against them.

A *simplified but expressive* topological score grammar keeps exactly the pieces that each source proved load-bearing and discards the rest:

- **From GTTM**: only the four structural *categories* (grouping, meter, time-span, prolongation) and the *form* of preference rules — soft, weighted, competing. The tonal well-formedness rules are dropped; blob/system evidence replaces musical evidence.
- **From Temperley**: incremental left-to-right optimisation with a switching cost — implemented once as `PreferenceRuleModule` (Viterbi over the candidate beam) and instantiated for grouping, meter and harmony (the key-finding extension = pitch-field finding).
- **From Hasty and London**: meter as projection, not lattice — `ElasticClock` pulls onsets toward the grid with a strength governed by the blobs' Kuramoto synchronisation, inside London's tactus/subdivision/bar bounds. Nothing is hard-quantised.
- **From Huron and Meyer**: the expectation pair (surprisal, conditional entropy) computed by `MarkovExpectation` over the symbolised macrostate; surprisal spikes tension, conditional entropy sets ambiguity; hazard drives audible-as-coming behaviour (Tension phase); the harmony module's posterior mass on non-current candidates is the Imagination phase (pre-sounding).
- **From Cohn and Lewin**: the harmonic field is a governed walk on the Tonnetz; governors emit transformations (P/L/R words), never chords; tonal cadence is replaced by return-along-the-parsimonious-path when the lamp returns toward equilibrium.
- **From Gjerdingen and Toussaint/Tidal**: a 12-frame schema lexicon with first-order succession, realised as patterns in a Tidal-style algebra (pattern = function from time-span to events) over Euclidean primitives.
- **From Di Scipio and Xenakis**: every blob generates and governs (softmax role weights, no governor class); density ceiling and texture archetype are governed statistical parameters, not per-note decisions.

What was deliberately left out for v4.2: Schenkerian voice-leading reduction beyond depth = material class; IDyOM-style variable-order contexts (the Markov layer is first-order with an open alphabet, extensible); learned rule weights (all weights are hand-set JSON, live-editable); explicit handover/crossfade machinery (removed by design, section 3).

2 Programming a dynamic range of sound with SuperCollider + Python

The system already splits responsibilities correctly: SuperCollider owns audio-rate synthesis and per-parameter smoothing (SynthDefs with linear VarLag glides), Python owns decision, timing and control. v4.2 widens the sonic range along four axes, all reachable from the existing python-osc client:

- **Voice families.** Sustained `\mpeVoice` (MPE polysynth), gated/percussive `\mpeHit` (new: one self-freeing node per hit, same timbre family so hits and pads blend), gestural `\bassStutter`, and the new `\preSound` pad for pre-sounding the predicted triad. New families are one SynthDef plus one scheduler mapping away.
- **Timing.** Sustained voices update every frame with `/n_set`; pattern hits are sent as timestamped OSC *bundles* from Python's look-ahead scheduler, so timing is sample-accurate on the server while the clock stays elastic and projective on the Python side (no SC TempoClock).
- **Continuous timbre space.** DETUNE / SHIMMER / NOISE / DRIVE / FMODE already give a roughness axis; the governor's roughness target and tension are summed into these before smoothing, so sensory dissonance is a governed quantity independent of pitch content.
- **Harmonic field.** Pitch gravitation toward the current field is applied in `_compute_targets` with a pull weakened for high-prominence voices (licensed dissonance) — the same mechanism as the EDIT-mode Hz offset, one more summed layer.

Further options not yet wired (all additive): SC-side *buses* for global filters/reverb driven by tension; `Pbind`-free but `Pmono`-like per-slot streams if latency budgets change; NRT rendering of a logged governor session for the paper's fixed-media examples; a Pi 5 headless mode using scsynth only.

3 Design change: crossfades are a byproduct, not a mechanism

The earlier binding policy ("crossfade control streams over several seconds, smoothed control signals") is removed. No module contains ramp, fade or handover code. Continuity arises from three probabilistic sources:

- **Softmax role weights.** Every contributor (each blob, and system-level virtual signals) carries a continuous weight per role, $w_i = \exp(s_i/T) / \Sigma_j \exp(s_j/T)$. A blob that dies has its score EMA decay; its weight decays with it and the others' rise — the handover is the normalisation.
- **Viterbi posteriors.** Each discrete module exposes softmax(V/T) over its beam; consumers that blend by posterior get gradual structural change: a boundary posterior grows over several frames before commitment, the harmony posterior leaks onto the next triad before the step (audible as pre-sounding).
- **Particle-filter estimates.** The meter period/phase estimate is a weighted mean over particles and drifts continuously; there is nothing to snap.

Tests `gv_roles_decay_without_election` and `pr_particle_filter_tracks` verify the behaviour without any fade code present.

4 Weighted rule scoring, weight files, elastic clock

Per module: candidate score $s_i(t) = \Sigma_r w_r \cdot f_r(\text{evidence}_t, c_i)$, $f_r \in [0,1]$. Discrete beam: $V_i(t) = [\log s_i(t) + \max_j(V_j(t-1) - \text{cost}(j \to i))] \cdot \text{forget}$, cost = switch_cost for $j \neq i$. Posterior = softmax(V/T). Continuous (meter): particles scored by the same rules, weights $\propto$ score, systematic resampling when ESS < N/2, Gaussian jitter. The committed tactus period is the hand-set mix (1–pf_mix)·beam + pf_mix·particle estimate (pf_mix in the JSON).

`governor_weights.json` — hand-set initial weights, read at start (versioned; user edits to a current-version file are never overwritten). `governor_runtime.json` — rewritten every 5 s (and on every EDIT-surface change) with live weights, committed candidates, posteriors, particle estimates/spread, role weights and prominence. `governor_log.txt` — one JSON line per write summarising the same. EDIT surface: `SET_PARAM` events whose param starts with `gov.` are consumed by the governor (`gov.grouping.rules.saddle`, `gov.roles.temperature`, `gov.meter.pf_mix` …); MIDI learn-maps can bind them to EC4 knobs unchanged.

Elastic clock: $t' = t + k\cdot(\text{grid}(t) - t)$, $k = 0.15 + 0.85\cdot$Kuramoto order parameter; tempo changes preserve phase; London bounds 0.4–2.0 s tactus, ≥ 0.1 s subdivision, ≤ 6 s bar; Hasty projection realised/denied by observed IOIs.

Table A — Pattern protocol, primitives, transformations, query patterns

Name	Signature	Semantics	Kind
PROTOCOL / TYPES (pattern_language.py)			
Span	`Span(start, end)`	Time span in cycles (one cycle = one bar). cycles(), intersect().	core
Event	`Event(start, end, value, velocity, degree, tags)`	A HIT: discrete onset with duration, accent [0,1], optional scale degree, free tags.	core
Pattern	`query(span) -> [Event]`	The protocol. Any object with query() is a pattern (Tidal's model: function from span to events).	core
FnPattern	`FnPattern(fn, label); p >> tf`	Pattern from a query function; fluent transform application with >>.	core
PRIMITIVES			
silence	`silence()`	No events.	primitive
pure	`pure(value, velocity)`	One whole-cycle event per cycle.	primitive
steps	`steps([1,0,1,1])`	Subdivide each cycle into equal slots; falsy = rest.	primitive
euclid	`euclid(k, n, rotation)`	E(k,n): k hits as evenly as possible over n slots (Bjorklund/Toussaint); E(3,8) = tresillo.	primitive
from_events	`from_events(events, period)`	Explicit events repeating every period (schemas realise into this).	primitive
sequence	`sequence(p1, p2, ...)`	One pattern per cycle in turn (Tidal cat).	primitive
stack	`stack(p1, p2, ...)`	Superimpose.	primitive
TRANSFORMATIONS (registry; Pattern -> Pattern; composable via compose() and apply_spec())			
fast	`fast(...)`	Speed up by factor (density up): fast(2) plays two cycles per cycle.	registered
slow	`slow(...)`	Slow down by factor (density down).	registered
rotate	`rotate(...)`	Shift the pattern earlier by `steps/n` of a cycle (Tidal `<~`).	registered
rev	`rev(...)`	Reverse each cycle.	registered
degrade	`degrade(...)`	Randomly drop hits with probability p (deterministic per event position + seed).	registered
mask	`mask(...)`	Keep only events overlapping a hit of the mask pattern (boolean AND).	registered
superimpose	`superimpose(...)`	Play the transformed copy on top of the original (stack(p, f(p))).	registered
every	`every(...)`	Apply f on every n-th cycle (offset selects which).	registered
iter	`iter(...)`	Rotate progressively: cycle c is shifted by c/n (Tidal `iter`).	registered
palindrome	`palindrome(...)`	Alternate forward and reversed cycles.	registered
chunk	`chunk(...)`	Apply f to the k-th 1/n of the cycle, rotating k each cycle.	registered
swing	`swing(...)`	Delay every second subdivision by `amount` of a slot (0..1).	registered
ply	`ply(...)`	Repeat each hit n times within its own duration.	registered
off	`off(...)`	Superimpose a copy delayed by t cycles and transformed by f (instant canon).	registered
accent	`accent(...)`	Multiply velocity by the accent pattern's value at each onset (e.g. steps([1.0,0.6,0.8,0.6])).	registered

Name	Signature	Semantics	Kind
velocity	`velocity(...)`	Scale all velocities by k.	registered
transpose	`transpose(...)`	Add n to every event's degree (within the current field).	registered
invert	`invert(...)`	Mirror degrees around a pivot degree.	registered
arp	`arp(...)`	Assign successive degrees from a list to successive hits (arpeggio).	registered
tag	`tag(...)`	Attach a tag to every event.	registered
density	`density(...)`	Continuous density control: keep a fraction `d` of hits (rank-ordered by velocity).	registered
QUERY PATTERNS (how consumers pull events)			
look-ahead	`pattern.query(Span(c_now, c_now+lookahead))`	PatternScheduler renders 250 ms ahead each frame, remembers the rendered-to cycle per slot.	query
cycle-wise	`pattern.query(Span(c, c+1))`	Per-cycle queries used by every/iter/chunk/rev/density transforms.	query
windowed	`pattern.query(Span(a, b))`	Arbitrary spans (rendering fills, previews, analysis).	query
spec-driven	`apply_spec(p, [("degrade", 0.3), ("ply", 2)])`	Serializable syntax governors emit and the runtime JSON records.	query
blob pattern	`ScoreGovernor.pattern_for_blob(blob)`	E(k,n): area -> k, persistence -> n, beat phase -> rotation; spec from tension/prominence/boundary.	query
schema pattern	`ScoreGovernor.schema_patterns()`	(melody, bass) patterns of the current schema; ornament level = tension.	query

Table B — Pattern-language layers, schema lexicon, frames

Layer / frame	Origin / mechanism	Melody degrees	Bass degrees	Footprint & tension	Description
LAYERS					
L0 rhythmic	euclid + transforms per blob	Each blob owns E(k,n); governors apply a spec (degrade/velocity/ply/fast). Gated \mpeHit.			
L1 schema	12 frames, (melody, bass) degree contours	Blob instantiates and ornaments the current schema; degrees resolve through the pitch field.			
L2 succession	SchemaMarkov first-order table	Row reweighted by tension/ambiguity/events (bias by tag); next() at grouping boundaries.			
L3 governance	ScoreState -> spec + bias	Tension, prominence rank, beat phase, boundary and events select transforms and reweight succession.			
SCHEMA LEXICON (schema_lexicon.py) — schema = (melodic contour, bass contour, footprint, tension profile)					
prinner	gjerdingen	[6, 5, 4, 3]	[4, 3, 2, 1]	4 beats; T=[0.6, 0.5, 0.4, 0.2]	Descending 6-5-4-3 over 4-3-2-1; the galant 'riposte'. Relaxation frame. Tags: descending, resolving
romanesca	gjerdingen	[1, 5, 6, 3]	[1, 5, 6, 3]	4 beats; T=[0.3, 0.4, 0.5, 0.35]	Leaping bass by descending thirds (1-5-6-3); stately opening gambit. Tags: opening, stable
fonte	gjerdingen	[4, 3, 3, 2]	[7, 1, 6, 7]	4 beats; T=[0.7, 0.5, 0.6, 0.4]	Sequential down-a-step pair (minor then major); post-boundary descent. Tags: sequence, descending
cadence	gjerdingen	[3, 2, 7, 1]	[1, 4, 5, 1]	4 beats; T=[0.4, 0.7, 0.9, 0.0]	Converging cadence: outer voices close onto the root; the arrival gesture. Tags: closing, resolving
plume	original	[1, 2, 3, 5, 6, 8]	[1, 1, 1, 3, 3, 5]	6 beats; T=[0.2, 0.35, 0.5, 0.65, 0.8, 0.9]	Rising contour with accelerating rhythm: a blob ascending. Tags: rising, tension
sink	original	[8, 6, 5, 3, 2, 1]	[5, 3, 3, 1, 1, 1]	6 beats; T=[0.8, 0.65, 0.5, 0.35, 0.2, 0.1]	Falling contour, decelerating: a blob sinking to rest. Tags: descending, resolving
merge	original	[5, 4, 3, 1]	[1, 2, 3, 1]	4 beats; T=[0.6, 0.7, 0.5, 0.1]	Two lines converging to a unison: blob merge. Tags: converging, resolving
split	original	[1, 2, 3, 5]	[1, 7, 6, 4]	4 beats; T=[0.2, 0.4, 0.6, 0.7]	Unison diverging into two lines: blob split. Tags: diverging, tension
stall	original	[3, 4, 3, 2, 3]	[1, 1, 1, 1, 1]	5 beats; T=[0.4, 0.5, 0.4, 0.5, 0.4]	Oscillation around one degree over a pedal: a blob hovering at turnaround. Tags: static, neighbour
overturn	original	[1, 3, 5, 6, 5, 3, 1]	[1, 1, 3, 4, 3, 1, 1]	7 beats; T=[0.2, 0.4, 0.6, 0.8, 0.6, 0.4, 0.15]	Rise-then-fall arc: a full convection overturn. Tags: arc, cycle

Layer / frame	Origin / mechanism	Melody degrees	Bass degrees	Footprint & tension	Description
plateau	original	[5, None, 5, None]	[1, None, None, 1]	4 beats; T=[0.3, 0.3, 0.3, 0.3]	Sparse sustained tones: the lamp at rest / low activity. Tags: static, sparse
swarm	original	[3, 5, 4, 6, 5, 7, 6, 8]	[1, 1, 2, 2, 3, 3, 4, 4]	8 beats; T=[0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85]	Dense, climbing cluster: many blobs active at once (Xenakian mass). Tags: dense, tension

Base succession table (rows sum to 1; governors multiply tagged columns and renormalise): immediate repeats ×0.35; resolving→tension ×1.6; tension→resolving ×1.6; closing→opening ×2.0; rising→descending ×1.3. Example row for **cadence**: romanesca 0.14, plume 0.11, split 0.11, swarm 0.11, prinner 0.07, fonte 0.07 …

Table C — System state summary, Markov model, tonic / chord / distance

Element	Definition / computation	Measures	Feeds / consumer
MACROSTATE SCALARS (SystemSummary; the 8-vector that is symbolised)			
occupancy_entropy	Shannon entropy of the area-weighted occupancy grid (8×6 over the ROI), normalised	wax spatial disorder	symbol digit; tension term; BOCPD signal
persistence_entropy	entropy of normalised H0 lifetimes	topological evenness	texture spread; tension term
coherence	Vicsek order parameter |Σ v■| / N over blob velocities	collective motion order	symbol digit; BOCPD signal
ke_total	Σ ½·area·speed²	activity level	symbol digit (ke/ke_ref); z-score on log(1+KE); BOCPD
w_equilibrium	marginal-EMD between current occupancy and the equilibrium grid	distance from rest	distance-from-tonic (physical half); harmony direction gradient
w_diagram	2-Wasserstein between consecutive persistence diagrams (Hungarian w/ diagonal; gudhi/POT if present)	rate of topological change	harmony step budget; regime boundary evidence
blob_count	tracked blobs	population	symbol digit; meter cardinality
tree_depth	max depth of the corrected merge tree	hierarchy depth	grouping role score; material class
EXTRAS (computed each frame; feed governors, not the symbol)			
per-blob kinematic	y, vy histories per id; phase & period via zero-crossings	oscillation	meter bank; role scores (periodicity, permutation entropy)
relational	pairwise distance mean/var, nearest-neighbour clustering, collinearity (SVD ratio), convex hull area, centre of mass, Kuramoto order over vertical-oscillation phases	constellation	boundary evidence; elastic strength; texture
aggregate physical	KE, vertical flux (area-weighted vy), coherence, COM height, mean speed	whole-lamp	tension; register centre; symbol
topological	diagram, persistence entropy, tree stats (leaf count, internal count, depth, Colless)	structure	prominence; grouping; texture
distributional	occupancy grid; vertical-velocity histogram (9 bins)	distributions	EMD terms of summary_distance()
permutation entropy	ordinal-pattern entropy of KE and flux windows (order 3)	regularity	prolongation role score; extras
MARKOV EXPECTATION MODEL (MarkovExpectation over the symbol alphabet)			
symbol	mixed-radix digits: blob_count {1 | 2–3 | 4+}, coherence terciles, KE terciles, flux sign, occupancy-entropy terciles → ≤ 243 symbols (open alphabet)	coarse-grained macrostate	row/column of the transition counts
surprisal	–log■ P(symbol_t | symbol_t–1) with Dirichlet smoothing	unexpectedness	tension term; harmony size licence; DRIVE
conditional entropy	H(next | current) per state	local unpredictability	ambiguity → pitch-field cardinality, density ceiling, gravitation
entropy rate	Σ π(s) H(next | s)	global unpredictability	runtime state (future: global governance gain)
pred. next entropy	Σ P(t|s) H(next|t)	one-step-ahead uncertainty	runtime state (future: handover softness)
recurrence	summary_distance(now, past) minimum beyond a lag	form-level return	nearest_past_state() (future: reprise trigger)
TONIC / CHORD / DISTANCE / DISSONANCE (non-tonal definitions implemented)			

Element	Definition / computation	Measures	Feeds / consumer
tonic	home triad (default C major) paired with the equilibrium occupancy grid	home region	TonnetzWalker.home; SystemStateTracker.equilibrium
chord	current triad on the Tonnetz + added tones (b7, 9, 11) as ambiguity rises	pitch field	ScoreState.pitch_field; gravitation in _compute_targets
distance from tonic	P/L/R path length to home (harmonic) + w_equilibrium (physical): one governed quantity	far from home ⇔ far from rest	ScoreState.distance_home; direction rule
dissonance	roughness proxy on the pitch-class set (Plomp–Levelt-shaped interval table) blended with tension	sensory roughness	roughness_target → DETUNE / NOISE
tension	smoothed composite: w·surprisal + w·w_equilibrium + w·KE + w·cond. entropy (weights in JSON)	prolongational scalar	TENSION source; schema bias; step size
AVAILABLE BUT NOT IMPLEMENTED			
transfer entropy	directional influence between blob streams	coupling	prominence (c) currently uses |corr(vy, flux)|
RQA / Lyapunov	recurrence rate, determinism, laminarity; divergence rate	chaoticity	offline notebooks first
exact 2-D EMD	POT / LP on occupancy grids	replace marginal-EMD	drop-in for emd_grid_marginal
variable-order Markov (PPM)	IDyOM-style longer contexts	expectation	same symbol stream
hazard from lifecycle logs	survival model conditioned on configuration	schedule	currently gap / mean-gap
HMM over macrostates	hidden regimes with emission = summary	regime posterior	would feed handover softness

Table D — Softmax continuous weighting and initialised weights

$w_i = \exp(s_i/T) / \Sigma_j \exp(s_j/T)$ over all contributors of a role (blob ids and virtual signals sys:ke, sys:flux, sys:occupancy_entropy, sys:kuramoto, sys:tree, sys:equilibrium). Scores are EMA-smoothed (0.85/0.15); absent contributors decay ×0.9 per frame. T = roles.temperature (0.6): T→0 is hard election, T large is uniform sharing. Role scores: grouping = merge-tree root proximity; meter = periodicity strength of the blob's y history; timespan = prominence; prolongation = permutation entropy (erraticness); harmony = slowness. Virtual signals are scaled by roles.virtual_signal_bias (0.5).

Weight path (governor_weights.json)	Initial value
roles.temperature	0.6
roles.virtual_signal_bias	0.5
roles.init.grouping	1.0
roles.init.meter	1.0
roles.init.timespan	1.0
roles.init.prolongation	1.0
roles.init.harmony	1.0
grouping.switch_cost	1.5
grouping.forget	0.7
grouping.temperature	0.5
grouping.commit_threshold	0.55
grouping.rules.gap	1.5
grouping.rules.change	2.0
grouping.rules.saddle	2.0
grouping.rules.changepoint	2.5
grouping.rules.regime	1.0
grouping.rules.relational	1.0
grouping.rules.coincidence	1.5
grouping.rules.regularity	1.5
meter.switch_cost	3.0
meter.forget	0.98
meter.temperature	1.0
meter.pf_mix	0.5
meter.rules.periodicity	2.0
meter.rules.london_prior	1.0

Weight path (governor_weights.json)	Initial value
meter.rules.cardinality	1.0
meter.rules.ratio_evidence	1.0
meter_pf.n_particles	150
meter_pf.rules.onset_alignment	2.0
meter_pf.rules.pf_continuity	1.0
meter_pf.rules.pf_london	0.5
timespan.smoothing	0.9
timespan.rules.integrated_persistence	1.0
timespan.rules.tree_depth	1.0
timespan.rules.coupling	0.8
timespan.rules.recurrence	0.6
timespan.rules.energy	0.6
timespan.rules.surface	1.2
prolongation.smoothing	0.92
prolongation.rules.surprisal	1.0
prolongation.rules.w_equilibrium	1.0
prolongation.rules.ke	0.8
prolongation.rules.cond_entropy	0.8
harmony.switch_cost	2.0
harmony.forget	0.95
harmony.temperature	0.8
harmony.step_rate_gain	4.0
harmony.rules.parsimony	2.0
harmony.rules.step_gate	2.0
harmony.rules.direction	1.5
harmony.rules.size_license	1.0
harmony.rules.content_fit	1.0
bocpd.hazard	0.0083
bocpd.signals	['ke', 'occupancy_entropy', 'flux', 'pair_dist_var', 'coherence']

Table E — Governor modules, candidate beams, rules

Module	Type / rule	Beam of candidates / rule semantics	Weight
grouping	GroupingModule (Viterbi, beam 2)	{kind: boundary} vs {kind: continue}; boundary commits when committed & posterior > commit_threshold & refractory 1.5 s; strength = normalised score; depth from saddle strength/score	
	`gap`	GPR2 analog: long quiet since the last blob/system event, then an event.	1.5
	`change`	GPR3 analog: abrupt change in an aggregate attribute (z-scored).	2.0
	`saddle`	Merge-tree saddle event: strength = absorbed branch persistence (normalized).	2.0
	`changepoint`	BOCPD posterior on any monitored signal (one mechanism, many signals).	2.5
	`regime`	Entropic/regime: conditional-entropy spike; topological: diagram-Wasserstein spike.	1.0
	`relational`	Relational: pairwise-distance variance shift, collinearity event, COM midline crossing.	1.0
	`coincidence`	Coincidence bonus: independent cues aligned within the window.	1.5
	`regularity`	Regularity / refractory: penalize a boundary too soon after the last one.	1.5
meter	MeterModule (Viterbi, beam 5)	{period, beats} for the top-3 elected periods × beats ∈ {2,3,4} (+ the incumbent); committed period mixed with the particle estimate	
	`periodicity`	Periodicity strength of the elected signal at the candidate's period.	2.0
	`london_prior`	London prior: tactus near 0.6-0.8 s preferred.	1.0
	`cardinality`	Blob-count cardinality evidence: n bodies -> n-beat grouping.	1.0
	`ratio_evidence`	Period-ratio evidence: a 3:2 ratio between two blobs' cycles favours 3- or 6-beat bars.	1.0
meter_pf	MeterParticleFilter (150 particles)	{period ∈ [0.4,2.0], phase} continuous; scored against blob turnaround onsets; jitter 0.02 s / 0.03	
	`onset_alignment`	Event-onset alignment: blob turnaround events near the particle's beat grid.	2.0
	`pf_continuity`	Continuity with the discrete beam's committed period.	1.0
	`pf_london`	London prior on the particle period.	0.5
harmony	HarmonyModule (Viterbi, beam 5)	current triad + P/L/R neighbours (+ PL/LP/PR/RP/LR/RL when tension or surprisal > 0.5); commit = Tonnetz step	
	`parsimony`	Parsimony: fewer/smaller voice-leading moves from the current field.	2.0
	`step_gate`	Step gate: moving is licensed only when the topological change budget is spent.	2.0
	`direction`	Direction: away from home when leaving equilibrium, toward home when returning.	1.5
	`size_license`	Size licensing: compound moves only under high tension / surprisal.	1.0
	`content_fit`	Fit to recently sounded pitch content (chord tones present in the texture).	1.0
timespan	prominence scorer (weighted sum, EMA)	no beam: (a) integrated persistence (b) tree depth (c) coupling |corr(vy, flux)| (d) recurrence (e) energy integral (f) surface: strong beat + chord tone	
	`integrated_persistence`		1.0
	`tree_depth`		1.0

Module	Type / rule	Beam of candidates / rule semantics	Weight
	`coupling`		0.8
	`recurrence`		0.6
	`energy`		0.6
	`surface`		1.2
prolongation	tension composite (weighted sum, EMA)	no beam: surprisal, w_equilibrium, KE, conditional entropy; ambiguity = H/3; hazard = gap/mean gap	
	`surprisal`		1.0
	`w_equilibrium`		1.0
	`ke`		0.8
	`cond_entropy`		0.8
POTENTIAL MODULES NOT IMPLEMENTED			
register	Viterbi over register bands	candidates: {low, mid, high, split}; evidence: COM height, hull, tree balance	
texture	beam over archetypes	{even polyphony, melody+accompaniment, mass/cloud}; evidence: Colless, persistence entropy, blob count	
schema succession as PRM	replace Markov sampling	candidates: next-schema; rules: tag fit to tension, footprint vs phrase length, recurrence	
density (Xenakis)	particle filter	continuous events/s; evidence: blob count, KE, ambiguity	
form / recurrence	beam over section labels	A/B/A′ recall from nearest_past_state()	
POTENTIAL CANDIDATE ATTRIBUTES			
grouping	boundary depth as a candidate field; boundary-at-lag candidates (t–Δ); per-boundary gesture class		
meter	accent profile (non-isochronous 3+3+2); subdivision; swing amount; phase as discrete candidate		
harmony	mode/quality bias; register/inversion; cardinality as a candidate field instead of a post-hoc rule		
meter_pf	period ratio particles (polyrhythm), tempo drift rate		

Table F — Signal election (elect signals, not blobs)

Signal	Drawn from	Analysis	What it decides	Impl.
IMPLEMENTED — meter role candidate bank (periodicity = autocorrelation peak; lag folded into 0.4–2.0 s by octave division)				
ke	aggregate: Σ ½·area·speed²	periodicity_strength over last 256 frames	candidate period + strength × (0.5 + role weight)	yes
flux	aggregate: area-weighted vertical velocity	same	the convection overturn: master period	yes
occupancy_entropy	distributional	same	breathing of spatial disorder	yes
kuramoto	relational: synchronisation order	same; also sets elastic strength k = 0.15 + 0.85·R	meter strength (grid vs elastic)	yes
com_y	relational: centre-of-mass height	same	bulk oscillation period	yes
per-blob y	kinematic	zero-crossing period per blob	candidate periods; period ratio of the two slowest blobs → 3:2 favours 3/6-beat bars	yes
blob turnarounds	kinematic events	onset list (direction changes)	particle-filter alignment evidence	yes
IMPLEMENTED — other roles				
prolongation ← sys:*	spectral flatness of ke/flux/occupancy/kuramoto	flattest signal carries the erratic role	tension governance weight	yes
grouping ← sys:tree	merge-tree depth > 1	constant bias	structure exists	yes
harmony ← sys:equilibrium	w_equilibrium gradient	direction of the Tonnetz walk	toward/away from home	yes
POTENTIAL — not implemented				
relative phase (pairs)	relational	Hilbert / zero-crossing phase difference	strong-beat placement; metric tension	no
elongation breathing	geometry	autocorrelation of elongation history	subdivision evidence	no
transfer entropy hub	information	directional influence graph	prominence / role weight	no
persistence entropy cycle	topological	periodicity of persistence entropy	texture pulse	no
spectral (FFT) peak	any	peak-to-flatness ratio instead of autocorrelation	alternative strength metric	no

Table G — Files and integration points

File	Status	Contents / hook
system_state.py	new	SystemSummary, entropies, EMD/Wasserstein, BOCPD, MarkovExpectation, SystemStateTracker, periodicity utilities
preference_rules.py	new	Rule/RuleRegistry, PreferenceRuleModule (Viterbi), ParticleFilterModule, WeightStore, softmax_weights
pattern_language.py	new	Span/Event/Pattern, primitives, 21 registered transforms, apply_spec, PatternScheduler (OSC bundles)
schema_lexicon.py	new	12 schemas, SchemaMarkov, realize(), degree_to_freq
harmony_tonnetz.py	new	Triad/P/L/R, shortest paths, TonnetzWalker, roughness proxy
elastic_clock.py	new	ElasticClock (projective, Kuramoto-governed pull, London bounds)
score_governor.py	new	ScoreState, rules, modules, roles, boundary evidence, merge-tree consumption, JSON I/O, EDIT hook, stamping
governor_weights.json	new	hand-set initial weights (versioned)
blob_synth_hits.scd	new	\mpeHit and \preSound SynthDefs (evaluate after blob_synth.scd)
routing.py	patched (diff)	8 governor Sources (TENSION, PROMINENCE_RANK, PHRASE_POSITION, BEAT_PHASE, SURPRISAL, AMBIGUITY, DEPTH, ROLE_WEIGHT) read from stamped blob attributes
voice_manager.py	patched (diff)	score_state / score_gain attributes; one additive layer in _compute_targets: gravitation, prominence mixer, roughness, boundary attack; reports sounding freq
audible_lava.py	patched (diff)	ScoreGovernor construction; merge tree on detection frames when governor is on; governor.update per frame; gov.* SET_PARAM routing; --no-governor flag
test_score_grammar.py	new	21 behavioural tests (pattern laws, schema Markov, entropies/BOCPD/Markov, Tonnetz, clock, hysteresis, particle filter, softmax, governor end-to-end, routing/VM hooks, performance)